AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
i-CREDIT HOURS ENGINEERING PROGRAMS

Tiny Telemetry Protocol: Efficient Periodic Telemetry over UDP with Optional Batching

CSE361: Computer Networking
Submitted to: Prof/Ayman Bahaa, Dr/Kareem Emara

Submitted by:

1. Abdelrahman Adel 23P0144
2. Monzer Mahmoud 23P0122
3. Omar Ahmed Abouraia 23P0100
4. Hossam Osama 23P0010
5. Zeina Reda 23P0181
6. Logine Mohamed 23P0187

Abstract

TinyTelemetry is a lightweight telemetry protocol designed for periodic sensor data reporting in resource-constrained environments such as IoT deployments.

The protocol operates over UDP to minimize transport overhead and to provide predictable, deterministic timing behavior.
This report presents the protocol architecture, packet format, implementation details, and an experimental evaluation focusing on efficiency, batching behavior, and robustness under packet loss.
Results demonstrate that TinyTelemetry significantly reduces per-report overhead for low-frequency reporting scenarios while maintaining simple and analyzable behavior under loss.

1. Introduction

Telemetry systems are a fundamental component of modern distributed and embedded systems.
In many IoT scenarios, devices periodically report sensor readings to a centralized collector under strict constraints on bandwidth, power consumption, and processing capability.

Traditional transport protocols such as TCP introduce connection establishment overhead, retransmission-induced jitter, and head-of-line blocking, which are undesirable for periodic telemetry.
TinyTelemetry is designed as a purpose-built protocol that trades strong reliability guarantees for simplicity, low overhead, and deterministic timing.

2. Protocol Design and Methodology

TinyTelemetry defines a stateless, unidirectional client-to-server communication model over UDP.
Each message consists of a fixed-size 12-byte header followed by a payload containing one or more sensor readings.

Sequence numbers are used to detect packet loss, duplication, and reordering without requiring acknowledgments or retransmissions.
Optional batching allows multiple readings to be transmitted in a single packet, amortizing header overhead and improving efficiency.

3. Implementation Details

The client implementation generates synthetic sensor readings and transmits them periodically according to a configurable reporting interval.
Batching and checksum features can be enabled at runtime.

The server maintains per-device state, parses incoming packets, detects anomalies based on sequence numbers, and logs metrics such as bytes per report, duplicate flags, and CPU processing time to CSV files for offline analysis.

4. Experimental Setup

Experiments were conducted using three reporting intervals (1 s, 10 s, and 30 s).
Packet loss was injected using the Linux netem traffic control module at controlled rates.

Metrics collected include average bytes per report, duplicate rate, inferred packet loss, and server-side CPU processing time.

5. Results and Interpretation

Figure 1 illustrates the relationship between reporting interval and average bytes per report.
As the reporting interval increases, more readings are batched per packet, significantly reducing overhead.

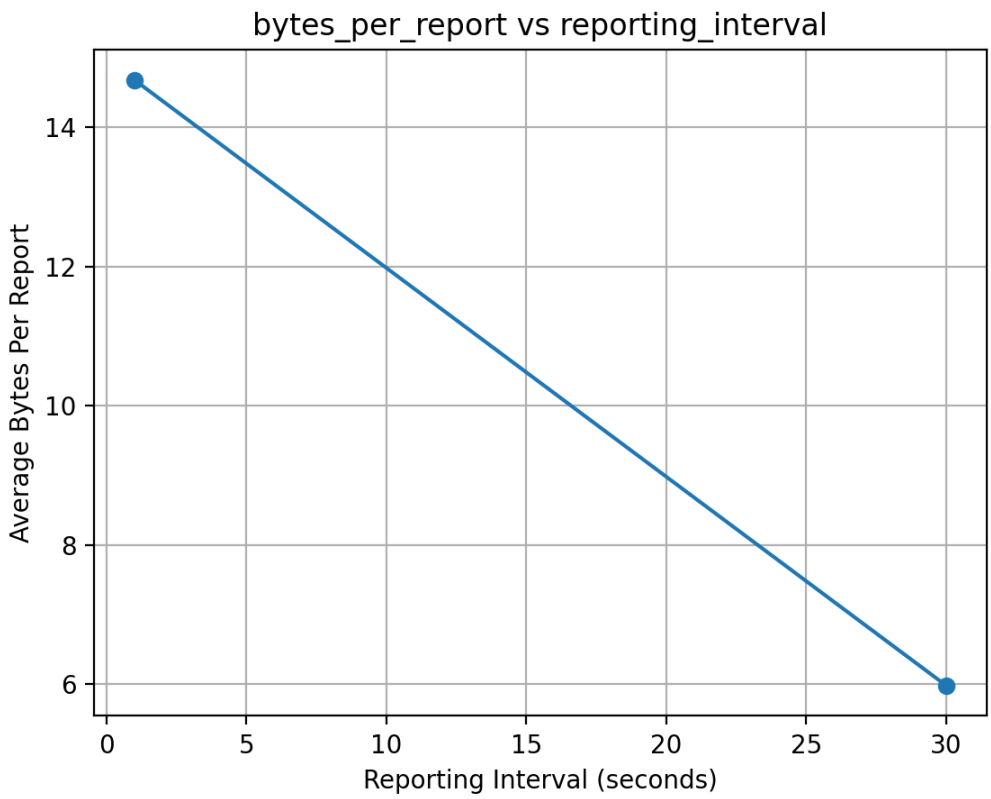

Figure 1: Average bytes per report vs. reporting interval

Figure 2 shows the observed duplicate rate as a function of packet loss.
The decrease in duplicate rate with increasing loss is consistent with the protocol's non-retransmitting UDP design.

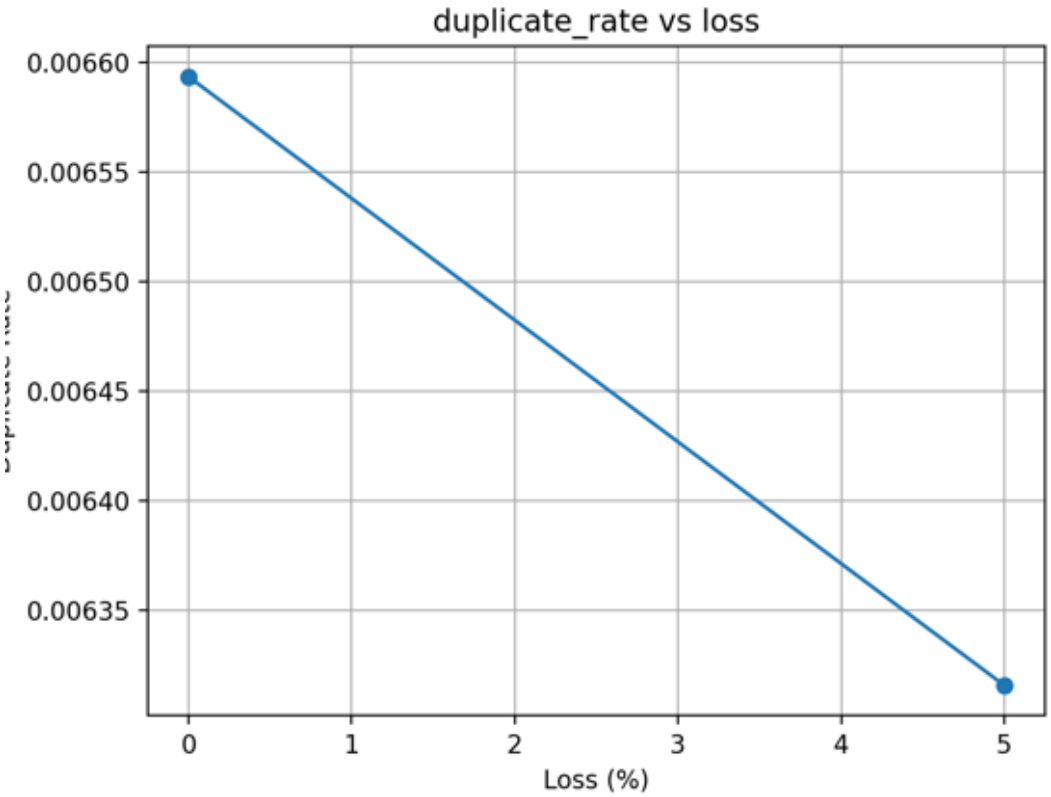

Figure 2: Duplicate rate vs. packet loss

6. Limitations and Future Work

TinyTelemetry intentionally omits reliability mechanisms to preserve deterministic timing and minimal overhead.
As a result, lost packets cannot be recovered, and the protocol is unsuitable for applications requiring guaranteed delivery.

Future work may explore optional lightweight reliability and security extensions, adaptive batching strategies, and validation on real embedded hardware.

7. References

[1] J. Postel, “User Datagram Protocol,” RFC 768, IETF, 1980.
[2] C. Bormann et al., “MQTT for Sensor Networks (MQTT-SN),” OASIS Standard, 2013.
[3] C. Perkins et al., “RTP: A Transport Protocol for Real-Time Applications,” RFC 3550, IETF, 2003.
[4] Linux Foundation, “Network Emulation (netem) Documentation.”
[5] Python Software Foundation, socket and struct Modules.